Diffusion Policy · arXiv 2303.04137

Stakeholder

18+3 · seat 2

Canonical continuous-action BC
before VLAs

Sep 21 paper club · VLA Architectures

Spine: chunk–commit–replan (T_p / T_a) — NOT classical planner

Chi et al. · RSS 2023 / IJRR extended

Where the value lives

Action head + control loop — not semantics / VLM

Canonical BC

Continuous-action recipe before VLAs
+46.9% relative

Control loop

chunk · commit · replan
T_o/T_p/T_a defaults still inherited

Real-robot

Push-T 95%
sauce near-human
bimanual 55–75%

Latency OK

DDIM ~10 Hz
0.1 s on 3080
not kHz torque

Ask of leadership

Treat action-chunk generative policies + receding execution as infrastructure, not a one-off research trick.

2026 OpenPI / BEHAVIOR stacks inherit these control assumptions

Credibility numbers (remember these)

+46.9%

avg relative vs best baseline (15 tasks)

95%

real Push-T success IoU 0.80

0.99

Kitchen p4 (vs BET 0.44)

10 Hz

DDIM real-time 10–16 steps ~0.1 s

Not a VLM

- No language · no web pretrain · task-specific ResNet + diffusion
- Value = action head + closed-loop chunk controller
- Journal: + control theory §4.5 · vision ablations · 3 bimanual real tasks
- Club dependency: if you ship OpenPI $\pi_{0.5}$ / BEHAVIOR, you inherit T_p/T_a ontology

Chunk–Commit–Replan · the motion/control contract

NOT classical planner / Diffuser full-trajectory planning · actions-only, O-conditioned

OBSERVE

$T_o \approx 2$

encode O_t once
(FiLM / cross-attn)

PREDICT

$T_p \approx 16$

denoise $A^k \to A^0$
joint action chunk

COMMIT

$T_a \approx 8$

execute prefix only
then stop

REPLAN

shift + loop

new O_t · warm-start
optional

One control cycle (@ ~10 Hz cmds → 125 Hz interp)

$T_a = 1$

mode flicker / idle trap

$T_a \approx 8$

sweet spot (paper)

$T_a \approx T_p$

open-loop / sluggish

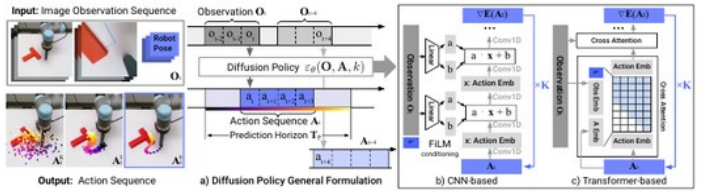


Figure 2. Diffusion Policy Overview a) General formulation. At time step t, the policy takes the latest T_o steps of observation data O_t as input and outputs T_a steps of actions A_t. b) In the CNN-based Diffusion Policy, FiLM (Feature-wise Linear Modulation) Perez et al. (2018) conditioning of the observation feature O_t is applied to every convolution layer, channel-wise. Starting from $\mathbf{A}_t^K$ drawn from Gaussian noise, the output of noise-prediction network ε_θ is subtracted, repeating K times to get $\mathbf{A}_t^0$, the denoised action sequence. c) In the Transformer-based Vaswani et al. (2017) Diffusion Policy, the embedding of observation $\mathbf{O}_t$ is passed into a multi-head cross-attention layer of each transformer decoder block. Each action embedding is constrained to only attend to itself and previous action embeddings (causal attention) using the attention mask illustrated.

2.2 DDPM Training

The training process starts by randomly drawing unmodified examples, $\mathbf{x}^0$, from the dataset. For each sample, we randomly select a denoising iteration k and then sample a random noise ε^k with appropriate variance for iteration k. The noise prediction network is asked to predict the noise from the data sample with noise added.

$$\mathscr{L} = MSE(\varepsilon^k, \varepsilon_\theta(\mathbf{x}^0 + \varepsilon^k, k)) \quad (3)$$

As shown in Ho et al. (2020), minimizing the loss function in Eq 3 also minimizes the variational lower bound of the KL-divergence between the data distribution $p(\mathbf{x}^0)$ and the distribution of samples drawn from the DDPM $q(\mathbf{x}^0)$ using Eq 1.

2.3 Diffusion for Visuomotor Policy Learning

While DDPMs are typically used for image generation ($\mathbf{x}$ is an image), we use a DDPM to learn robot visuomotor policies. This requires two major modifications in the formulation: 1. changing the output $\mathbf{x}$ to represent robot actions. 2. making the denoising processes *conditioned* on input observation $\mathbf{O}_t$. The following paragraphs discuss each of the modifications, and Fig. 2 shows an overview.

Closed-loop action-sequence prediction: An effective action formulation should encourage temporal consistency and smoothness in long-horizon planning while allowing prompt reactions to unexpected observations. To accomplish this goal, we commit to the action-sequence prediction produced by a diffusion model for a fixed duration before replanning. Concretely, at time step t the policy takes the latest T_o steps of observation data $\mathbf{O}_t$ as input and predicts T_p steps of actions, of which T_a steps of actions are executed on the robot without re-planning. Here, we define T_o as the observation horizon, T_p as the action prediction horizon and T_a as the action execution horizon. This encourages temporal action consistency while remaining responsive. More details about the effects of T_a are discussed in Sec 4.3. Our formulation also allows receding horizon control Mayne and Michalska (1988) to futher improve action smoothness by warm-starting the next inference setup with previous action sequence prediction.

Visual observation conditioning: We use a DDPM to approximate the conditional distribution $p(\mathbf{A}_t|\mathbf{O}_t)$ instead of the joint distribution $p(\mathbf{A}_t, \mathbf{O}_t)$ used in Janner et al. (2022a) for planning. This formulation allows the model to predict actions conditioned on observations without the cost of inferring future states, speeding up the diffusion process and improving the accuracy of generated actions. To capture the conditional distribution $p(\mathbf{A}_t|\mathbf{O}_t)$, we modify Eq 1 to:

$$\mathbf{A}_t^{k-1} = \alpha(\mathbf{A}_t^k - \gamma\varepsilon_\theta(\mathbf{O}_t, \mathbf{A}_t^k, k) + \mathcal{N}(0, \sigma^2 I)) \quad (4)$$

The training loss is modified from Eq 3 to:

$$\mathscr{L} = MSE(\varepsilon^k, \varepsilon_\theta(\mathbf{O}_t, \mathbf{A}_t^0 + \varepsilon^k, k)) \quad (5)$$

The exclusion of observation features $\mathbf{O}_t$ from the output of the denoising process significantly improves inference speed and better accommodates real-time control. It also helps to make **end-to-end** training of the vision encoder feasible. Details about the visual encoder are described in Sec. 3.2.

3 Key Design Decisions

In this section, we describe key design decisions for Diffusion Policy as well as its concrete implementation of ε_θ with neural network architectures.

3.1 Network Architecture Options

The first design decision is the choice of neural network architectures for ε_θ. In this work, we examine two common network architecture types, convolutional neural networks (CNNs) Ronneberger et al. (2015) and Transformers Vaswani et al. (2017), and compare their performance and training characteristics. Note that the choice of noise prediction network ε_θ is independent of visual encoders, which will be described in Sec. 3.2.

CNN-based Diffusion Policy We adopt the 1D temporal CNN from Janner et al. (2022b) with a few modifications: First, we only model the conditional distribution $p(\mathbf{A}_t|\mathbf{O}_t)$ by conditioning the action generation process on observation features $\mathbf{O}_t$ with Feature-wise Linear Modulation (FiLM)

Fig 2 · horizons T_o/T_p/T_a + CNN/Transformer

arXiv 2303.04137

Multimodal basins · Langevin commits, then holds

Push-T left/right · Kitchen order · BlockPush stage order

Diffusion Policy

commits to ONE basin per rollout

temporal consistency across T_p

BET / BC-RNN (foil)

mode flicker mid-horizon

independent per-step GMM/k-means

Club line to BEHAVIOR

DP chunk+replan → ACT → π_0 flow chunks → RTC soft-inpaint → BEHAVIOR (exec26/inpaint4)

What stayed vs what moved

STAYED
- generative continuous chunks
- execute-prefix / replan
- overlap consistency need

MOVED
- DDPM → flow matching
- sync → async soft-inpaint
- implicit corr → explicit Σ

Subsequent lineage · CLEAR citations only

Denoiser changed · chunk–commit–replan–overlap habit stayed

2023 — **Diffusion Policy** — DDPM chunks T_p/T_a replan

2023 — **ACT** — CVAE chunks (parallel)

2024 — **π_0** — cites Chi'23 DDPM→flow

2025 — **$\pi_{0.5}$ / OpenPI** — same chunk habit

2025 — **RTC + BEHAVIOR** — soft-inpaint 30/26/4

π_0 · 2410.24164

- Cites Chi et al. 2023
- Flow matching on PaliGemma
- Continuous chunks ~50 Hz
- Same control ontology

RTC · 2506.07339

- Soft-inpaint async chunks
- Freeze delay-safe prefix
- Works on diffusion OR flow
- No retrain

BEHAVIOR 1st · 2512.06951

- Pi0.5 + corr. noise
- predict 30 / exec 26 / inpaint 4
- q ~ 0.26
- NOT training DP itself

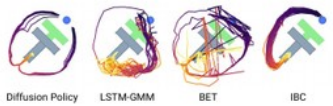


Figure 3. Multimodal behavior. At the given state, the end-effector (blue) can either go left or right to push the block. **Diffusion Policy** learns both modes and commits to only one mode within each rollout. In contrast, both **LSTM-GMM** Mandlekar et al. (2021) and **IBC** Florence et al. (2021) are biased toward one mode, while **BET** Shafiullah et al. (2022) fails to commit to a single mode due to its lack of temporal action consistency. Actions generated by rolling out 40 steps for the best-performing checkpoint.

speculate that there are two primary reasons for this discrepancy: First, action multimodality is more pronounced in position-control mode than it is when using velocity control. Because Diffusion Policy better expresses action multimodality than existing approaches, we speculate that it is inherently less affected by this drawback than existing methods. Furthermore, position control suffers less than velocity control from compounding error effects and is thus more suitable for action-sequence prediction (as discussed in the following section). As a result, Diffusion Policy is both less affected by the primary drawbacks of position control and is better able to exploit position control's advantages.

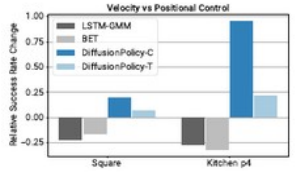


Figure 4. Velocity v.s. Position Control. The performance difference when switching from velocity to position control. While both BCRNN and BET performance decrease, Diffusion Policy is able to leverage the advantage of position and improve its performance.

4.3 *Benefits of Action-Sequence Prediction*

Sequence prediction is often avoided in most policy learning methods due to the difficulties in effectively sampling from high-dimensional output spaces. For example, IBC would struggle in effectively sampling high-dimensional action space with a non-smooth energy landscape. Similarly, BC-RNN and BET would have difficulty specifying the number of modes that exist in the action distribution (needed for GMM or k-means steps).

In contrast, DDPM scales well with output dimensions without sacrificing the expressiveness of the model, as demonstrated in many image generation applications. Leveraging this capability, Diffusion Policy represents action in the form of a high-dimensional action sequence, which naturally addresses the following issues:

- **Temporal action consistency**: Take Fig 3 as an example. To push the T block into the target from the bottom, the policy can go around the T block from either left or right. However, suppose each action in the sequence is predicted as independent multimodal distributions (as done in BC-RNN and BET). In that case, consecutive actions could be drawn from different modes, resulting in jittery actions that alternate between the two valid trajectories.

- **Robustness to idle actions**: Idle actions occur when a demonstration is paused and results in sequences of identical positional actions or near-zero velocity actions. It is common during teleoperation and is sometimes required for tasks like liquid pouring. However, single-step policies can easily overfit to this pausing behavior. For example, BC-RNN and IBC often get stuck in real-world experiments when the idle actions are not explicitly removed from training.

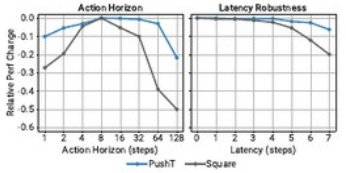


Figure 5. Diffusion Policy Ablation Study. Change (difference) in success rate relative to the maximum for each task is shown on the Y-axis. **Left**: trade-off between temporal consistency and responsiveness when selecting the action horizon. **Right**: Diffusion Policy with position control is robust against latency. Latency is defined as the number of steps between the last frame of observations to the first action that can be executed.

4.4 *Training Stability*

While IBC, in theory, should possess similar advantages as diffusion policies. However, achieving reliable and high-performance results from IBC in practice is challenging due to IBC's inherent training instability Ta et al. (2022). Fig 6 shows training error spikes and unstable evaluation performance throughout the training process, making hyperparameter turning critical and checkpoint selection difficult. As a result, Florence et al. (2021) evaluate every checkpoint and report results for the best-performing checkpoint. In a real-world setting, this workflow necessitates the evaluation of many policies on hardware to select a final policy. Here, we discuss why Diffusion Policy appears significantly more stable to train.

An implicit policy represents the action distribution using an Energy-Based Model (EBM):

$$p_\theta(\mathbf{a}|\mathbf{o}) = \frac{e^{-E_\theta(\mathbf{o},\mathbf{a})}}{Z(\mathbf{o},\theta)} \tag{6}$$

where $Z(\mathbf{o},\theta)$ is an intractable normalization constant (with respect to $\mathbf{a}$).

Fig 4–5 · pos/vel + T_a / latency

arXiv 2303.04137

To train the EBM for implicit policy, an InfoNCE-style loss function is used, which equates to the negative log-likelihood of Eq 6:

$$\mathscr{L}_{infoNCE} = -\log\left(\frac{e^{-E_\theta(\mathbf{o},\mathbf{a})}}{e^{-E_\theta(\mathbf{o},\mathbf{a})} + \sum_{j=1}^{N_{neg}} e^{-E_\theta(\mathbf{o},\tilde{\mathbf{a}}^j)}}\right) \quad (7)$$

where a set of negative samples $\{\tilde{\mathbf{a}}^j\}_{j=1}^{N_{neg}}$ are used to estimate the intractable normalization constant $Z(\mathbf{o},\theta)$. In practice, the inaccuracy of negative sampling is known to cause training instability for EBMs Du et al. (2020); Ta et al. (2022).

Diffusion Policy and DDPMs sidestep the issue of estimating $Z(\mathbf{a},\theta)$ altogether by modeling the **score function** Song and Ermon (2019) of the same action distribution in Eq 6:

$$\nabla_{\mathbf{a}} \log p(\mathbf{a}|\mathbf{o}) = -\nabla_{\mathbf{a}} E_\theta(\mathbf{a},\mathbf{o}) - \underbrace{\nabla_{\mathbf{a}} \log Z(\mathbf{o},\theta)}_{=0} \approx -\varepsilon_\theta(\mathbf{a},\mathbf{o}) \quad (8)$$

where the noise-prediction network $\varepsilon_\theta(\mathbf{a},\mathbf{o})$ is approximating the negative of the score function $\nabla_{\mathbf{a}} \log p(\mathbf{a}|\mathbf{o})$ Liu et al. (2022), which is independent of the normalization constant $Z(\mathbf{o},\theta)$. As a result, neither the inference (Eq 4) nor training (Eq 5) process of Diffusion Policy involves evaluating $Z(\mathbf{o},\theta)$, thus making Diffusion Policy training more stable.

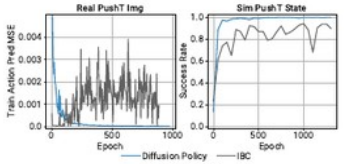


Figure 6. Training Stability. Left: IBC fails to infer training actions with increasing accuracy despite smoothly decreasing training loss for energy function. Right: IBC's evaluation success rate oscillates, making checkpoint selection difficult (evaluated using policy rollouts in simulation).

4.5 Connections to Control Theory

Diffusion Policy has a simple limiting behavior when the tasks are very simple; this potentially allows us to bring to bear some rigorous understanding from control theory. Consider the case where we have a linear dynamical system, in standard state-space form, that we wish to control:

$$\mathbf{s}_{t+1} = \mathbf{A}\mathbf{s}_t + \mathbf{B}\mathbf{a}_t + \mathbf{w}_t, \qquad \mathbf{w}_t \sim \mathcal{N}(0,\Sigma_w).$$

Now imagine we obtain demonstrations (rollouts) from a linear feedback policy: $\mathbf{a}_t = -\mathbf{K}\mathbf{s}_t$. This policy could be obtained, for instance, by solving a linear optimal control problem like the Linear Quadratic Regulator. Imitating this policy does not need the modeling power of diffusion, but as a sanity check, we can see that Diffusion Policy does the right thing.

In particular, when the prediction horizon is one time step, $T_p = 1$, it can be seen that the optimal denoiser which minimizes

$$\mathscr{L} = MSE(\varepsilon^k, \varepsilon_\theta(\mathbf{s}_t, -\mathbf{K}\mathbf{s}_t + \varepsilon^k, k)) \quad (9)$$

is given by

$$\varepsilon_\theta(\mathbf{s},\mathbf{a},k) = \frac{1}{\sigma_k}[\mathbf{a} + \mathbf{K}\mathbf{s}],$$

where σ_k is the variance on denoising iteration k. Furthermore, at inference time, the DDIM sampling will converge to the global minima at $\mathbf{a} = -\mathbf{K}\mathbf{s}$.

Trajectory prediction ($T_p > 1$) follows naturally. In order to predict $\mathbf{a}_{t+t'}$ as a function of $\mathbf{s}_t$, the optimal denoiser will produce $\mathbf{a}_{t+t'} = -\mathbf{K}(\mathbf{A}-\mathbf{B}\mathbf{K})^{t'}\mathbf{s}_t$; all terms involving $\mathbf{w}_t$ are zero in expectation. This shows that in order to perfectly clone a behavior that depends on the state, the learner must implicitly learn a (task-relevant) dynamics model Subramanian and Mahajan (2019); Zhang et al. (2020). Note that if either the plant or the policy is nonlinear, then predicting future actions could become significantly more challenging and once again involve multimodal predictions.

5 Evaluation

We systematically evaluate Diffusion Policy on 15 tasks from 4 benchmarks Florence et al. (2021); Gupta et al. (2019); Mandlekar et al. (2021); Shafiullah et al. (2022). This evaluation suite includes both simulated and real environments, single and multiple task benchmarks, fully actuated and under-actuated systems, and rigid and fluid objects. We found Diffusion Policy to consistently outperform the prior state-of-the-art on all of the tested benchmarks, with an average success-rate improvement of 46.9%. In the following sections, we provide an overview of each task, our evaluation methodology on that task, and our key takeaways.

5.1 Simulation Environments and datasets

Robomimic Mandlekar et al. (2021) is a large-scale robotic manipulation benchmark designed to study imitation learning and offline RL. The benchmark consists of 5 tasks with a proficient human (PH) teleoperated demonstration dataset for each and mixed proficient/non-proficient human (MH) demonstration datasets for 4 of the tasks (9 variants in total). For each variant, we report results for both state- and image-based observations. Properties for each task are summarized in Tab 3.

Push-T adapted from IBC Florence et al. (2021), requires pushing a T-shaped block (gray) to a fixed target (red) with a circular end-effector (blue)s. Variation is added by random initial conditions for T block and end-effector. The task requires exploiting complex and contact-rich object dynamics to push the T block precisely, using point contacts. There are two variants: one with RGB image observations and another with 9 2D keypoints obtained from the ground-truth pose of the T block, both with proprioception for end-effector location.

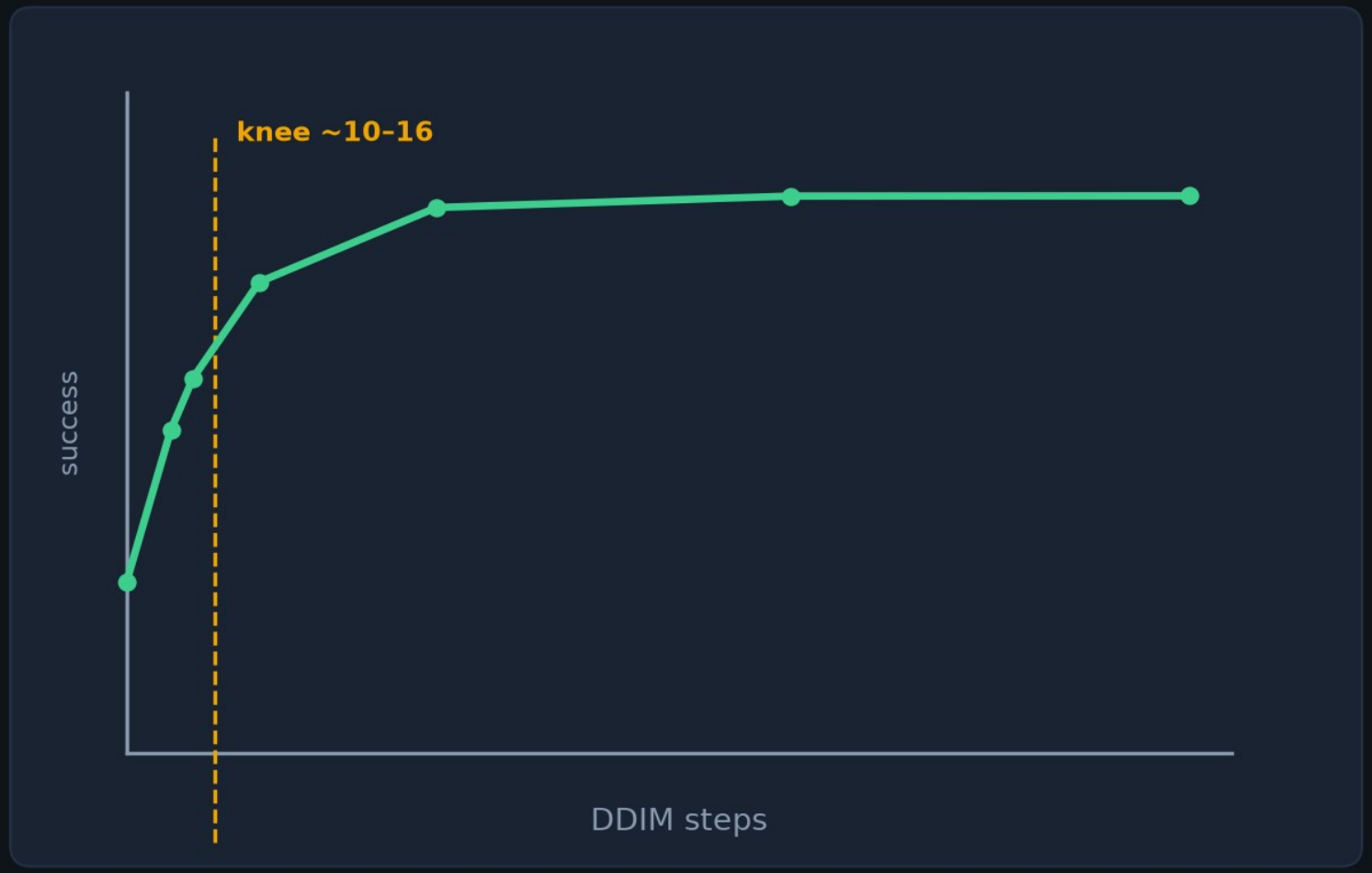
Multimodal Block Pushing adapted from BET Shafiullah et al. (2022), this task tests the policy's ability to model multimodal action distributions by pushing two blocks

Tables · state/vision results

arXiv 2303.04137

H-DDIM · quality saturates ~10–16 steps

Train 100 (iDDPM square-cosine) → infer DDIM 10–16 for real-time



Real-time budget

~0.1 s @ 10–16 steps

RTX 3080 (paper)

cmds 10 Hz → 125 Hz

P1 experiment

steps {4..100}
infer-only on ckpt

< 1 GPU-h

H-loop · closed-loop replan vs open-loop chunks

Same failure mode as long VLA chunks without replan

CLOSED-LOOP

$T_a < T_p$ · replan every commit

1. O_t → denoise chunk
2. exec T_a prefix
3. new O_{t+ta}
4. replan (warm-start)

OPEN-LOOP

$T_a \approx T_p$ · rare / no mid replan

1. O_t → denoise long chunk
2. exec almost all of T_p
3. obs may have shifted
4. recovery fails / lags

Empiricist test: set T_a=T_p and disable mid-episode replan; inject obs delay 0–4 steps

Expect clear gap at delay ≥2 — ancestor of VLA long-chunk brittleness

Q & A

Infrastructure, not a trick

Diffusion Policy · Sep 21

Diffusion Policy: Visuomotor Policy Learning via Action Diffusion

Cheng Chi*[1], Zhenjia Xu*[1], Siyuan Feng[2], Eric Cousineau[2], Yilun Du[3], Benjamin Burchfiel[2], Russ Tedrake [2,3], Shuran Song[1,4]

Abstract

This paper introduces Diffusion Policy, a new way of generating robot behavior by representing a robot's visuomotor policy as a conditional denoising diffusion process. We benchmark Diffusion Policy across 15 different tasks from 4 different robot manipulation benchmarks and find that it consistently outperforms existing state-of-the-art robot learning methods with an average improvement of 46.9%. Diffusion Policy learns the gradient of the action-distribution score function and iteratively optimizes with respect to this gradient field during inference via a series of stochastic Langevin dynamics steps. We find that the diffusion formulation yields powerful advantages when used for robot policies, including gracefully handling multimodal action distributions, being suitable for high-dimensional action spaces, and exhibiting impressive training stability. To fully unlock the potential of diffusion models for visuomotor policy learning on physical robots, this paper presents a set of key technical contributions including the incorporation of receding horizon control, visual conditioning, and the time-series diffusion transformer. We hope this work will help motivate a new generation of policy learning techniques that are able to leverage the powerful generative modeling capabilities of diffusion models. Code, data, and training details is available diffusion-policy.cs.columbia.edu

Keywords

Imitation learning, visuomotor policy, manipulation

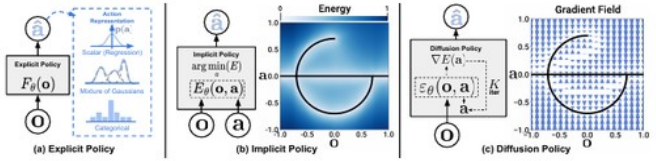


Figure 1. Policy Representations. a) Explicit policy with different types of action representations. b) Implicit policy learns an energy function conditioned on both action and observation and optimizes for actions that minimize the energy landscape c) Diffusion policy refines noise into actions via a learned gradient field. This formulation provides stable training, allows the learned policy to accurately model multimodal action distributions, and accommodates high-dimensional action sequences.

1 Introduction

Policy learning from demonstration, in its simplest form, can be formulated as the supervised regression task of learning to map observations to actions. In practice however, the unique nature of predicting robot actions — such as the existence of multimodal distributions, sequential correlation, and the requirement of high precision — makes this task distinct and challenging compared to other supervised learning problems.

Prior work attempts to address this challenge by exploring different *action representations* (Fig 1 a) – using mixtures of Gaussians Mandlekar et al. (2021), categorical representations of quantized actions Shafiullah et al. (2022), or by switching the *the policy representation* (Fig 1 b) – from explicit to implicit to better capture multi-modal distributions Florence et al. (2021); Wu et al. (2020).

In this work, we seek to address this challenge by introducing a new form of robot visuomotor policy that generates behavior via a "conditional denoising diffusion process Ho et al. (2020) on robot action space", **Diffusion Policy**. In this formulation, instead of directly outputting an action, the policy infers the action-score gradient, conditioned on visual observations, for K denoising iterations (Fig. 1 c). This formulation allows robot policies to inherit several key properties from diffusion models – significantly improving performance.

- **Expressing multimodal action distributions.** By learning the gradient of the action score function Song and Ermon (2019) and performing Stochastic Langevin Dynamics sampling on this gradient field, Diffusion policy can express arbitrary normalizable distributions Neal et al. (2011), which includes multimodal action distributions, a well-known challenge for policy learning.

Cover · Chi et al. · RSS/IJRR

arXiv 2303.04137